25/6/2026

Python Programming
Assignment - 8

1) Explain the steps to create and use a user-defined module.

Ans:

Step 1 : Create a Module
File : Marvellous.py

```
def Display():
    print("Jai Ganesh...")

PI = 3.14
```

Step 2 : Import and Use in Another File
File : main.py

```
import Marvellous

Marvellous.Display()
print("Value of PI : ", Marvellous.PI)
```

Output :
Jai Ganesh...
Value of PI : 3.14

This is how we import and use our own code written in other files.

2) Why is indentation mandatory in Python while it is optional in languages like C, C++ or Java?

Ans: In C, C++ or Java the compiler relies entirely on symbols. Semicolons (;) tell it where a statement ends, and curly braces {} tell it where a block of code begins and ends. Because the symbols do all the ~~thinking~~ talking, the compiler treats ~~spaces~~ spaces, tabs & newlines as completely invisible

But whereas in Python, it stripped away the braces and semicolons to reduce visual clutter. To replace them it uses colon (:) to introduce a new block and whitespaces to measure it. That's why indentation is mandatory in Python.

3) What is an IndentationError? When does Python raise this error?

Ans: An IndentationError in Python is a syntax error that occurs when the structural alignment of the code - specifically the spaces or tabs at the beginning of a line - voilates Python's formatting rules.

Python raises an IndentationError when its parser encounters incorrect, inconsistent, or misplaced whitespace at the beginning of a line of code. Because Python relies entirely on whitespace instead of curly braces {} to define code blocks, strict alignment is mandatory. This error is a subclass of SyntaxError and is caught during the parsing stage before your program actually begins running.

4) What are the different ways to import a module? Which one is recommended and why?

Ans: There are mainly 4 ways to import a module:

i) import module_name : This syntax import the whole module in the program. ~~All the content which~~

ii) from module_name import x : This syntax will import a specific object or function which will be required.

iii) from module_name import * : This syntax will import everything from the module. Usually this syntax is not recommended.

iv) ~~import~~ ~~improm~~ module_name as alias : This syntax ~~w~~ imports with alias name for shorter usage.

~~Mostly~~ From above options, we recomend the first one i.e. import module_name. Because, this imports the entire module, requiring you to access its contents using the module name and dot notation. It is highly explicit. Anyone reading the code immediately knowns ~~ex~~ exactly where fun is coming from, which prevents confusion and accidential overwriting of variables.

5) How does Python internally indentify the start and end of a code block using indentation?

Ans: Python internally identifies the start and end of a code block during the lexical analysis by tracking whitspace using an internal stack and generating two special tokens. These tokens acts exactly like the opening { and closing } curly braces found in languages like C, C++ or Java.

The Internal Mechanism:

When you run a Python file, the interpreter reads the source code line by line and processes the leading whitespaces at the beginning of each line. It manages block boundaries using data structure called the Indentation Stack.

6) Why is from module import * discouraged in professional code?

Ans: from module import * discouraged because it pollutes the namespace, makes code unreadable (you can't tell where names & come from), causes silent name collisions, creates tight coupling to a module's internals, and breaks static analysis tools. It trades short-term convenience for long-term maintainability problems.

7) How does modular programming improve code reusability, testing, and maintanance?

Ans: Modular programming improves reusability by allowing individual modules to be written once and used accross multiple projects without rewriting logic. For testing, each module can be tested in isolation, making it easier to identify and fix bugs without affecting the rest of the codebase. For maintainance, changes made to one module do not ripple accross the entire program, so updates & bug fixes are contained, clean, and manageable.

8) What happens internally when Python executes an import module_name statement?

Ans: Python first searches for the module. If not found, it locates the file using sys.path, compiles it to bytecode (.pyc), executes the module's code in a new namespace, & stores the resulting module object. Future imports of the same module simply return the cached object without re-executing the file.

9) What happens if a module contains print statements outside any function?

Ans: If a module contains print statements outside any function, those print statements execute immediatly at import time. Every time the module is imported anywhere in the program, the print output appears automatically. This is considered bad practice because importing a module should not produce side effects - it should only define functions, classes, and variables, leaving execution to the caller.

10) Explain why Python modules are considered the foundation of scalable software design.

Ans: Modules enforce seperation of concerns by dividing a program into discrete, self-contained units, each responsible for a specific functionally. This structure allows large teams to work independently on different parts of a codebase without conflicts. As a system grows, new modules can be added without disturbing existing ones, making the architecture naturally extensible, maintainable, and scalable over time.